

1º Bimestre - Atividade Aula 03

Gramática BNF e derivação de um comando Java

Aluno: Gabriel de Souza Carvalho | **RA:** 24027655-2

1. Código escolhido

Trecho com o laço **while** do Java, baseado no exemplo da W3Schools (https://www.w3schools.com/java/java_while_loop.asp). O programa imprime os números de 0 a 4.

```
public class ExemploWhile {  
    public static void main(String[] args) {  
        int i = 0;  
        while (i < 5) {  
            System.out.println(i);  
            i++;  
        }  
    }  
}
```

O trecho analisado pela gramática é o que está dentro do *main*: a declaração `int i = 0;` seguida do comando `while`.

2. Regras gramaticais (BNF)

Os símbolos entre < > são não terminais. O que aparece fora deles (palavras reservadas, parênteses, chaves, operadores e ponto e vírgula) são terminais. A gramática foi reduzida ao necessário para gerar o trecho acima.

R1	<trecho>	::= <declaracao> <comando_while>
R2	<declaracao>	::= <tipo> <identificador> = <literal_inteiro> ;
R3	<comando_while>	::= while (<condicao>) <bloco>
R4	<condicao>	::= <identificador> <op_relacional> <expressao>
R5	<bloco>	::= { <lista_comandos> }
R6	<lista_comandos>	::= <comando> <lista_comandos> <comando>
R7	<comando>	::= <comando_impressao> <comando_incremento>
R8	<comando_impressao>	::= System.out.println (<expressao>) ;
R9	<comando_incremento>	::= <identificador> ++ ;
R10	<expressao>	::= <identificador> <literal_inteiro>
R11	<op_relacional>	::= < <= > >= == !=
R12	<tipo>	::= int
R13	<identificador>	::= i
R14	<literal_inteiro>	::= 0 5

3. Derivação mais à esquerda

A derivação parte do símbolo inicial <trecho>. Em cada passo é substituído o não terminal mais à esquerda, usando a regra indicada na coluna da direita, até sobraem só terminais.

Passo	Forma sentencial	Regra
0	<trecho>	início
1	=> <declaracao> <comando_while>	R1
2	=> <tipo> <identificador> = <literal_inteiro> ; <comando_while>	R2
3	=> int <identificador> = <literal_inteiro> ; <comando_while>	R12
4	=> int i = <literal_inteiro> ; <comando_while>	R13
5	=> int i = 0 ; <comando_while>	R14
6	=> int i = 0 ; while (<condicao>) <bloco>	R3
7	=> int i = 0 ; while (<identificador> <op_relacional> <expressao>) <bloco>	R4
8	=> int i = 0 ; while (i <op_relacional> <expressao>) <bloco>	R13
9	=> int i = 0 ; while (i < <expressao>) <bloco>	R11
10	=> int i = 0 ; while (i < <literal_inteiro>) <bloco>	R10
11	=> int i = 0 ; while (i < 5) <bloco>	R14
12	=> int i = 0 ; while (i < 5) { <lista_comandos> }	R5
13	=> int i = 0 ; while (i < 5) { <comando> <lista_comandos> }	R6
14	=> int i = 0 ; while (i < 5) { <comando_impressao> <lista_comandos> }	R7
15	=> int i = 0 ; while (i < 5) { System.out.println (<expressao>) ; <lista_comandos> }	R8
16	=> int i = 0 ; while (i < 5) { System.out.println (<identificador>) ; <lista_comandos> }	R10
17	=> int i = 0 ; while (i < 5) { System.out.println (i) ; <lista_comandos> }	R13
18	=> int i = 0 ; while (i < 5) { System.out.println (i) ; <comando> }	R6
19	=> int i = 0 ; while (i < 5) { System.out.println (i) ; <comando_incremento> }	R7
20	=> int i = 0 ; while (i < 5) { System.out.println (i) ; <identificador> ++ ; }	R9
21	=> int i = 0 ; while (i < 5) { System.out.println (i) ; i ++ ; }	R13

A última linha contém apenas terminais e é exatamente a sequência de tokens do trecho Java, então o código pertence à linguagem gerada pela gramática.

4. Árvore de derivação (resumida)

```
<trecho>
|-- <declaracao>
|   |-- <tipo> ----- int
|   |-- <identificador> ----- i
|   |-- =
|   |-- <literal_inteiro> ----- 0
|   |-- `-- ;
|-- `-- <comando_while>
|   |-- while
|   |-- (
|   |-- <condicao>
|   |   |-- <identificador> ---- i
|   |   |-- <op_relacional> ---- <
|   |   |-- `-- <expressao> ----- <literal_inteiro> -- 5
|   |-- )
|-- `-- <bloco>
|   |-- {
|   |-- <lista_comandos>
|   |   |-- <comando> ----- <comando_impressao>
|   |   |   |
|   |   |   |   System.out.println ( i ) ;
|   |   |-- `-- <lista_comandos> -- <comando> -- <comando_incremento>
|   |   |   |
|   |   |   |   i ++ ;
|   |-- `-- }
```

5. Breve explicação

O **while** repete um bloco enquanto uma condição for verdadeira, testando a condição *antes* de cada volta. Diferente do **for**, ele não tem espaço próprio para inicializar e atualizar a variável de controle, por isso a gramática precisou de uma regra separada para a declaração (`int i = 0;`, antes do laço) e outra para o incremento (`i++;`, dentro do bloco). No exemplo, *i* começa em 0; como $0 < 5$, o bloco executa, imprime o valor e soma 1. Isso se repete até *i* valer 5, quando a condição fica falsa e o laço termina. A saída é 0, 1, 2, 3 e 4, uma por linha.

A regra R6 é recursiva: `<lista_comandos>` pode gerar um comando seguido de outra lista. É isso que permite ter qualquer quantidade de comandos dentro das chaves com um número finito de regras. A derivação mostra que o código não é aceito "por intuição": ele é válido porque dá para chegar nele a partir do símbolo inicial aplicando só as regras definidas. Se faltasse o ponto e vírgula depois de `i++`, por exemplo, nenhuma sequência de regras produziria aquela sentença, e o compilador acusaria erro de sintaxe.